



National University of Computer and Emerging Sciences

Generative Adversarial Networks

AI-4009 Generative AI

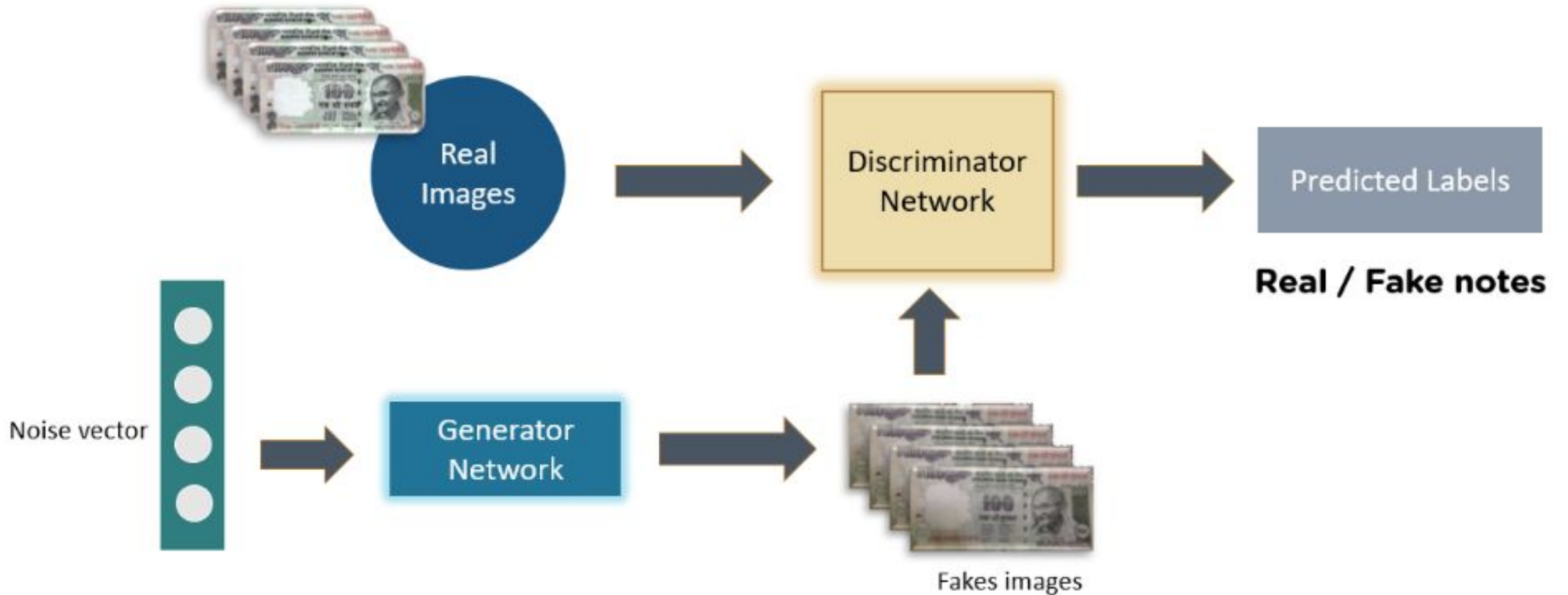
Dr. Akhtar Jamil

Department of Computer Science

Generative Adversarial Network (GAN)

- **Generative Adversarial Networks (GANs)** were introduced in 2014 by Ian J. Goodfellow and co-authors.
- GANs perform **unsupervised learning tasks** in machine learning.
- It consists of 2 models that automatically discover and learn the patterns in input data.
 - **Generator and Discriminator.**
- Can be used in **generating images, videos, text.**

Generative Adversarial Network (GAN)



Generative Adversarial Network (GAN)

Single Image Super-Resolution



(Ledig et al 2016)

DCGANs for LSUN Bedrooms



(Radford et al 2015)

Image to Image Translation

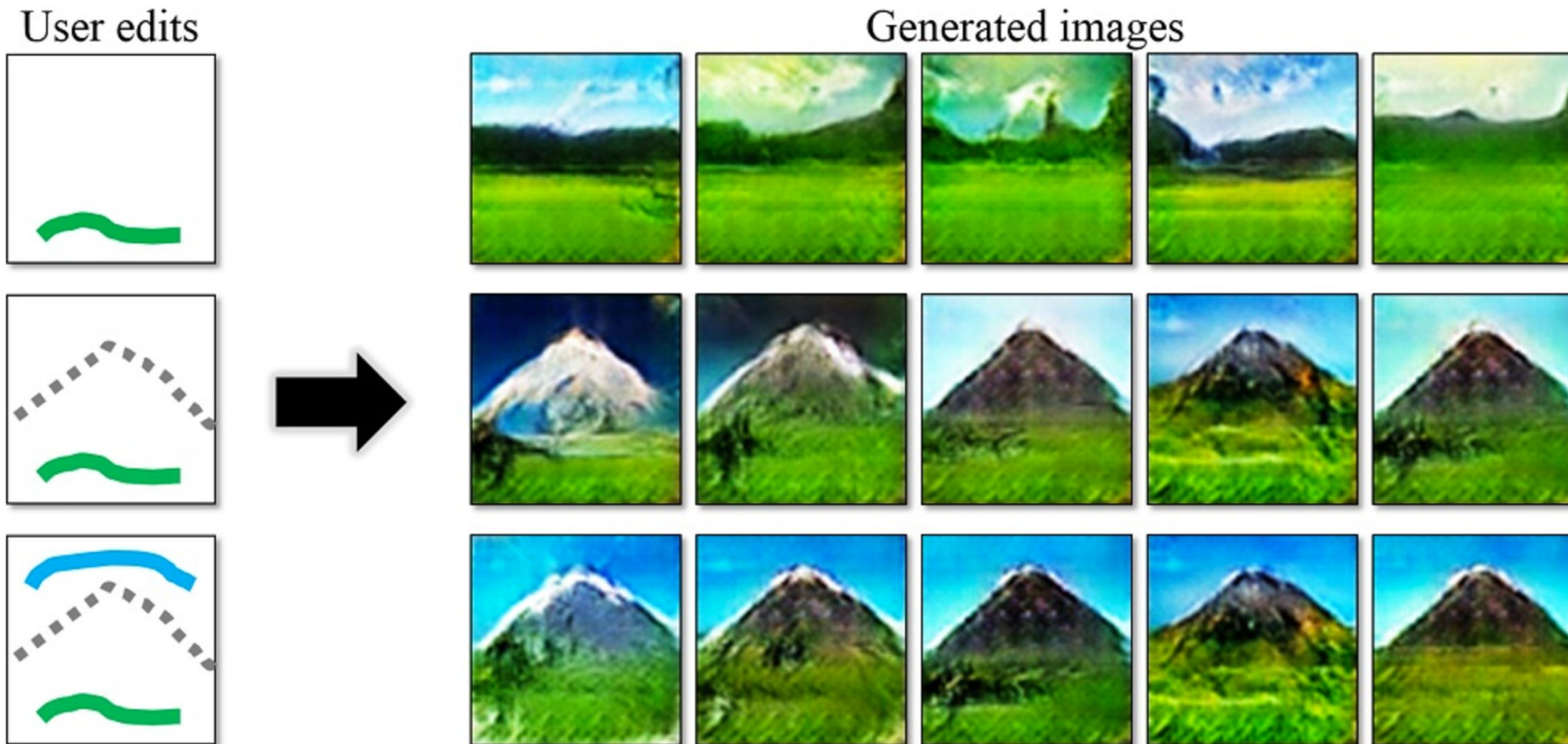


11:34 / 1:55:53

(Isola et al 2016)

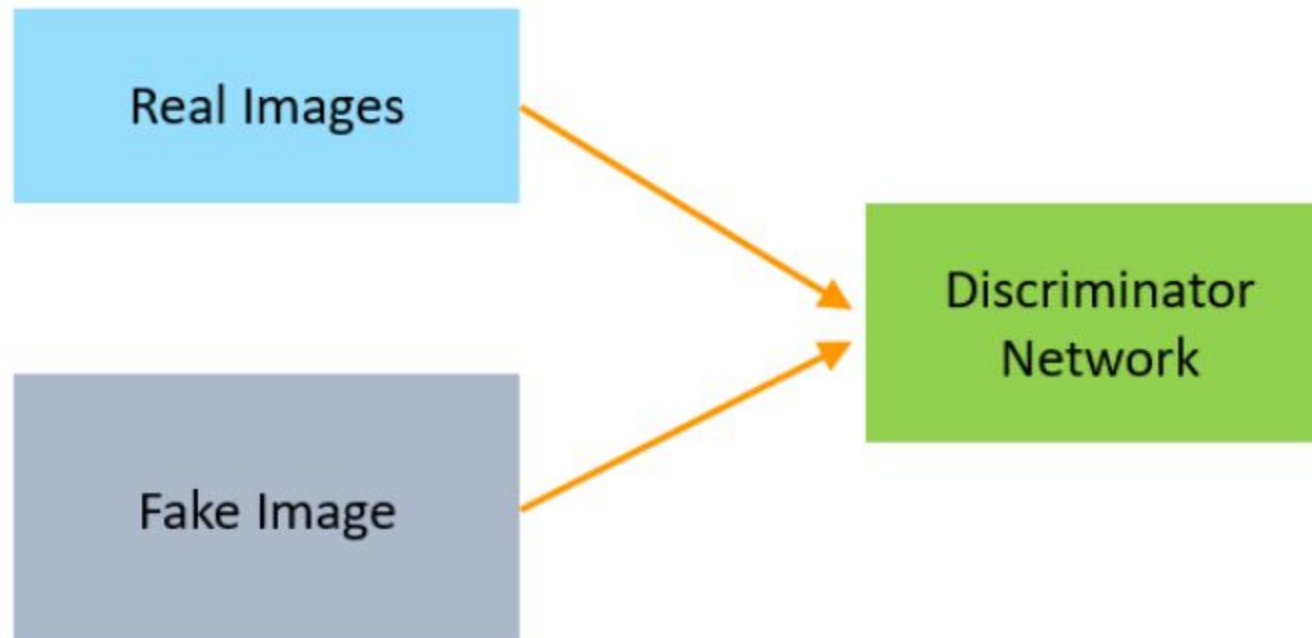
(Goodfellow 2016)

Magic of GANs...



Generative Adversarial Network

- Generator vs Discriminator
- They compete with each other to capture, and replicate the variations within a dataset.

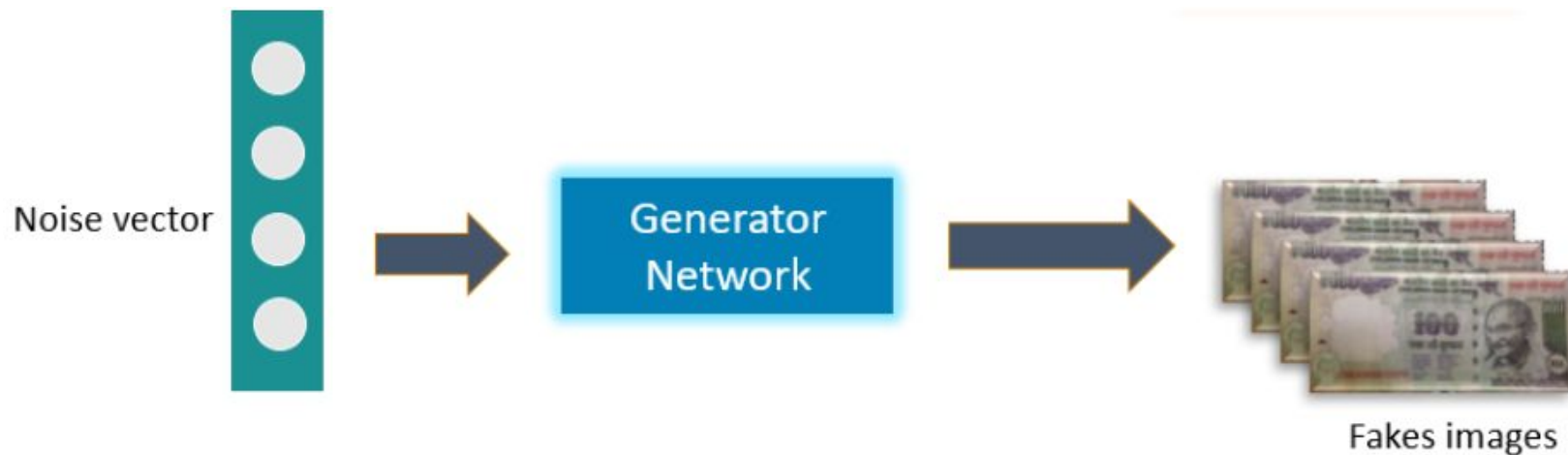


Generative Adversarial Network

- **Generator:**
- A Generator in GANs is a neural network that creates fake data to be trained on the discriminator.
- It learns to generate probable data.
- The generated examples/instances become negative training examples for the discriminator.
- It takes a fixed-length random vector carrying noise as input and generates a sample.

Generative Adversarial Network

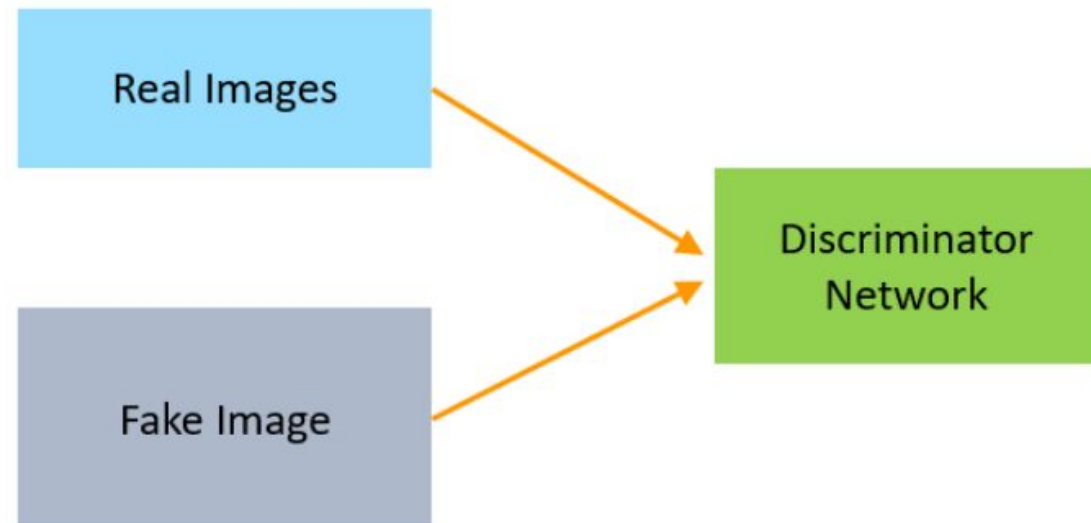
- The main aim of the **Generator** is to make the discriminator **classify its output as real**.
- This can happen when the generator **generates realistic images**.



Generative Adversarial Network

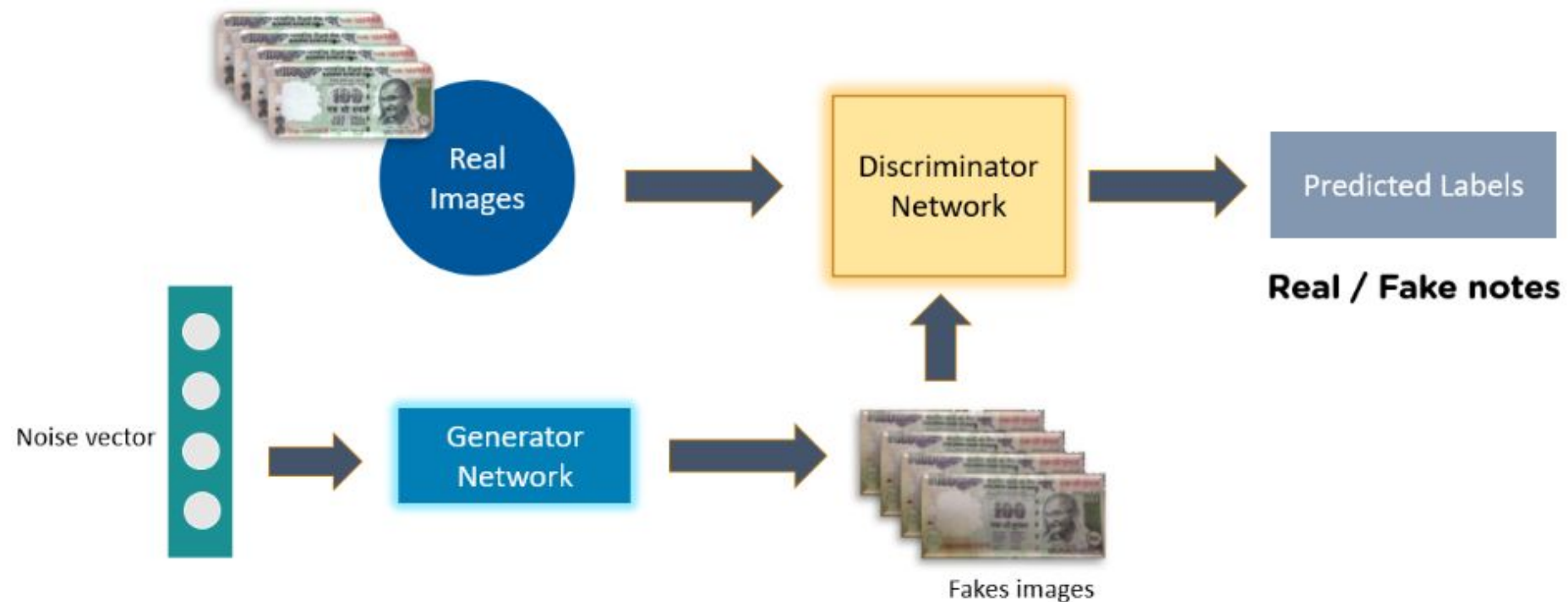
- **Discriminator**

- The Discriminator is a neural network that identifies real data from the fake data created by the Generator.
- The discriminator's training data comes from two different sources:
 - The **real data** instances
 - The **fake data** instances



Generative Adversarial Network

- Complete GAN Network



Random Input

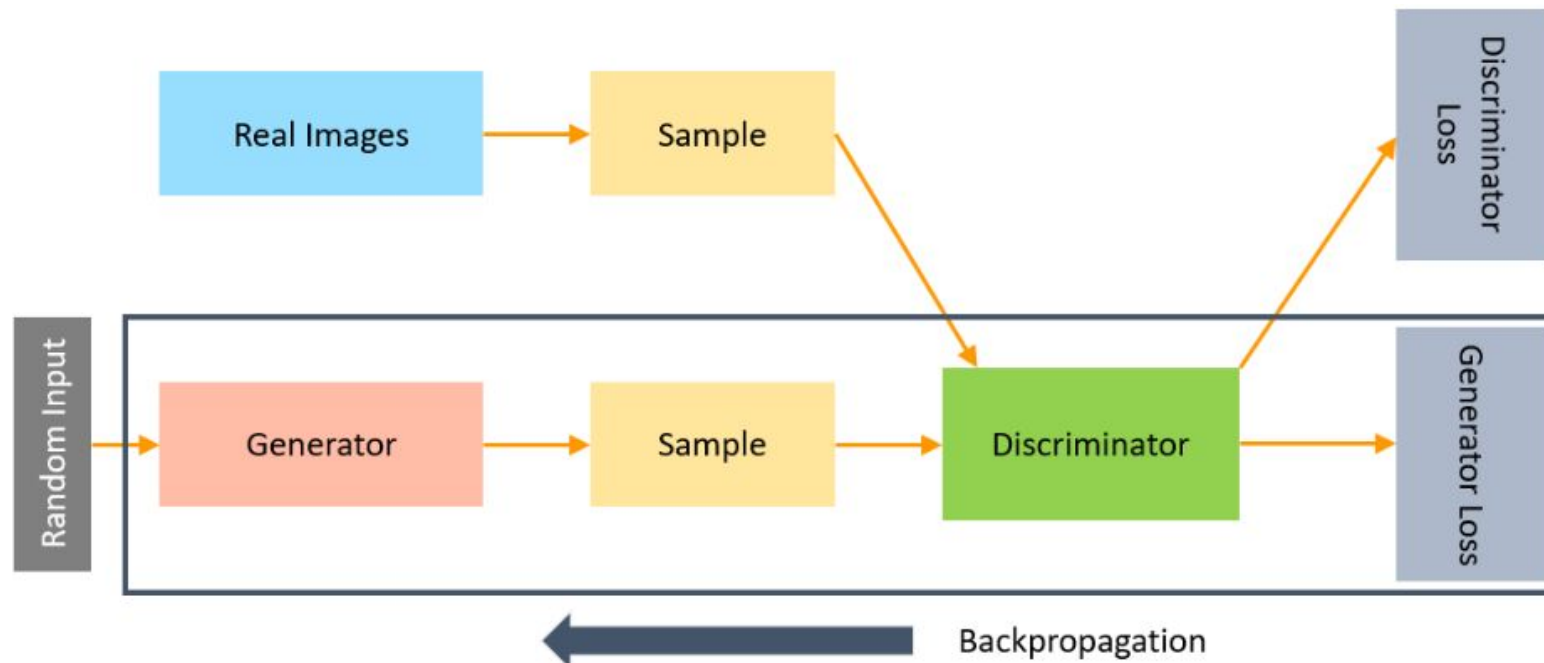
- In its most basic form, a GAN takes **random noise as its input**.
- The generator then **transforms this noise into a meaningful output**.
- **By introducing noise**, we can get the GAN to produce a wide variety of data.
- What choices?
- **Gaussian (Normal) Distribution:**
 - Sample values from a normal distribution with a mean of 0 and a standard deviation of 1 (i.e., $N(0,1)$).
- **Uniform Distribution:** The values can be sampled uniformly from a fixed range, e.g. **-1 and 1** or **0 and 1**.

GAN Training

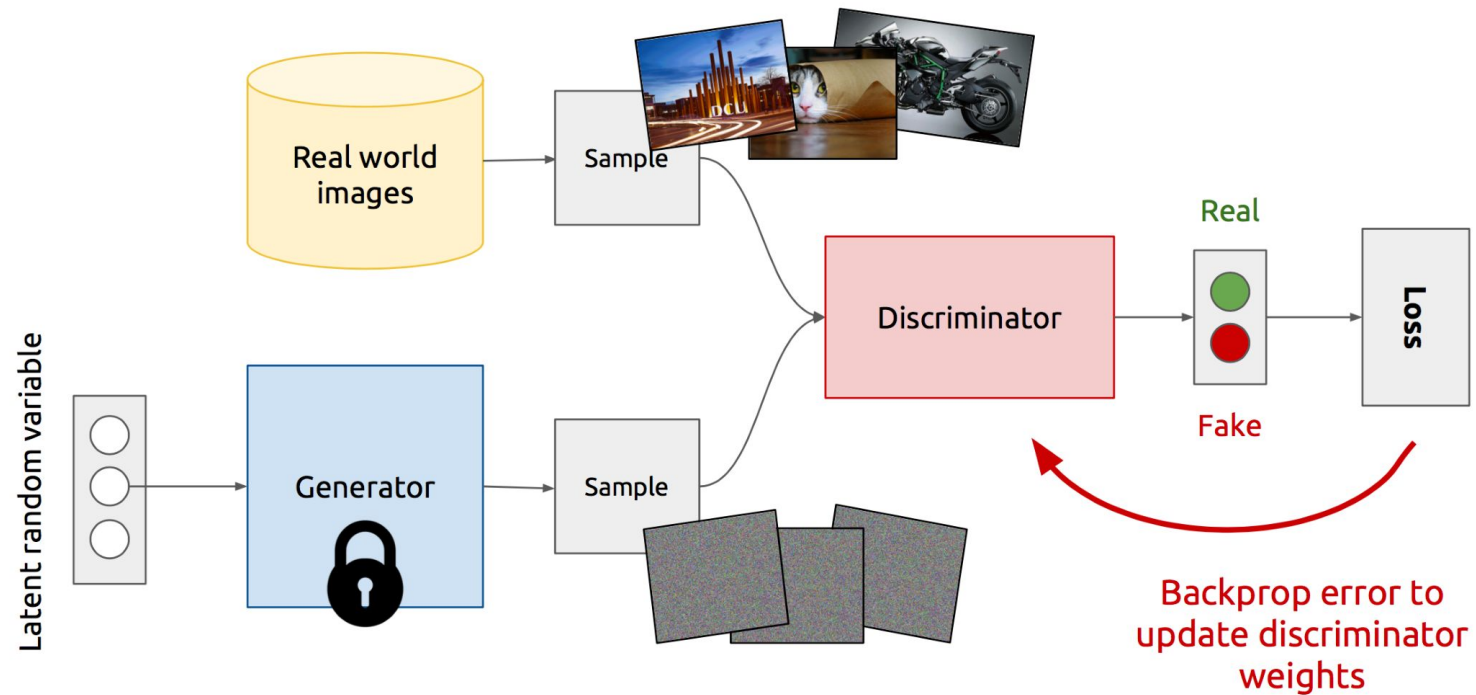
- Because a GAN contains two separately trained networks, its training algorithm must address two complications:
 - GANs must switch two different kinds of training (generator and discriminator).
 - GAN convergence is hard to identify.
- **Alternating Training**
 1. The discriminator trains for one or more epochs.
 2. The generator trains for one or more epochs.
 3. Repeat steps 1 and 2 to continue to train the generator and discriminator networks.

Generative Adversarial Network

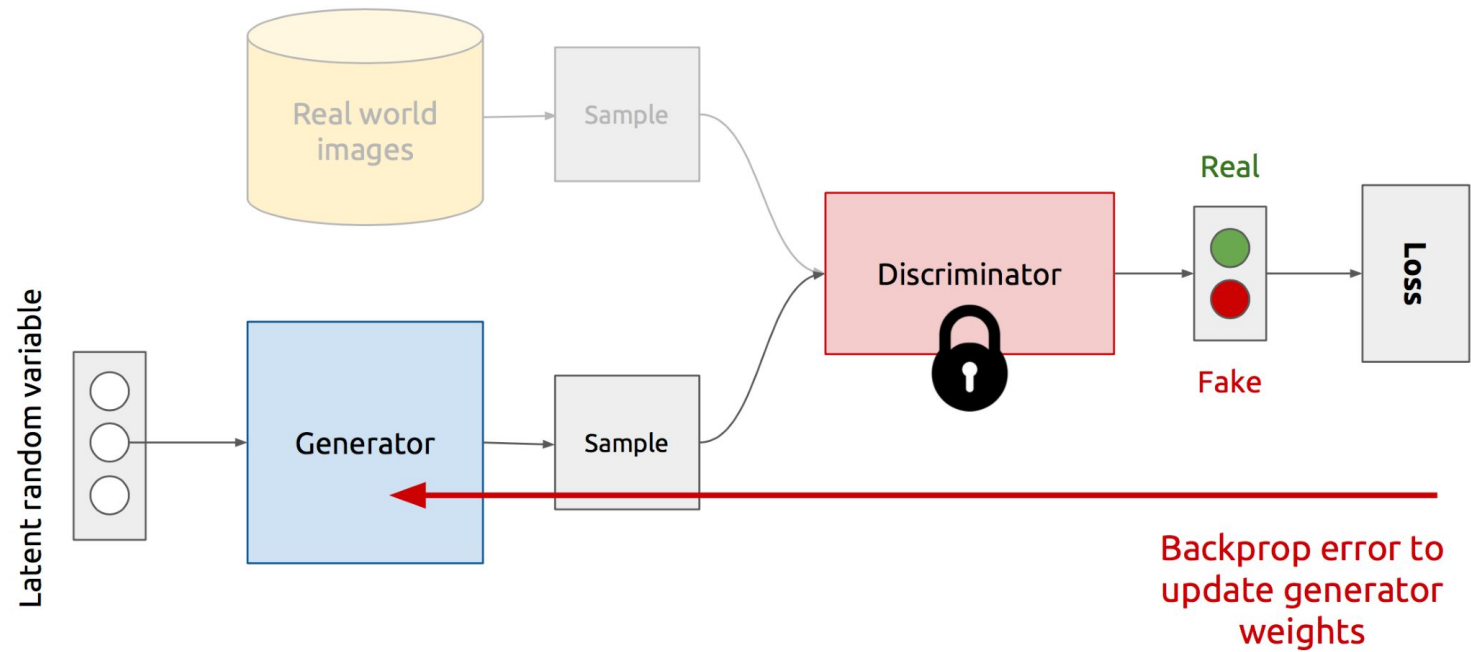
- The **backpropagation** method is used to adjust each weight in the right direction by calculating the weight's impact on the output.



Training Discriminator



Training Generator



Generative Adversarial Network

- **Loss Functions**
- GANs generate the probability distribution of a particular data.
- Loss functions measures distance between the **distribution of the data generated by the GAN and the distribution of the real data.**
 - Active research area
 - Many approaches have been proposed.
- **GANs have two loss functions:** one for generator training and one for discriminator training.
- Two common GAN loss functions:
 - **Minimax loss**
 - **Wasserstein loss**

Loss Functions

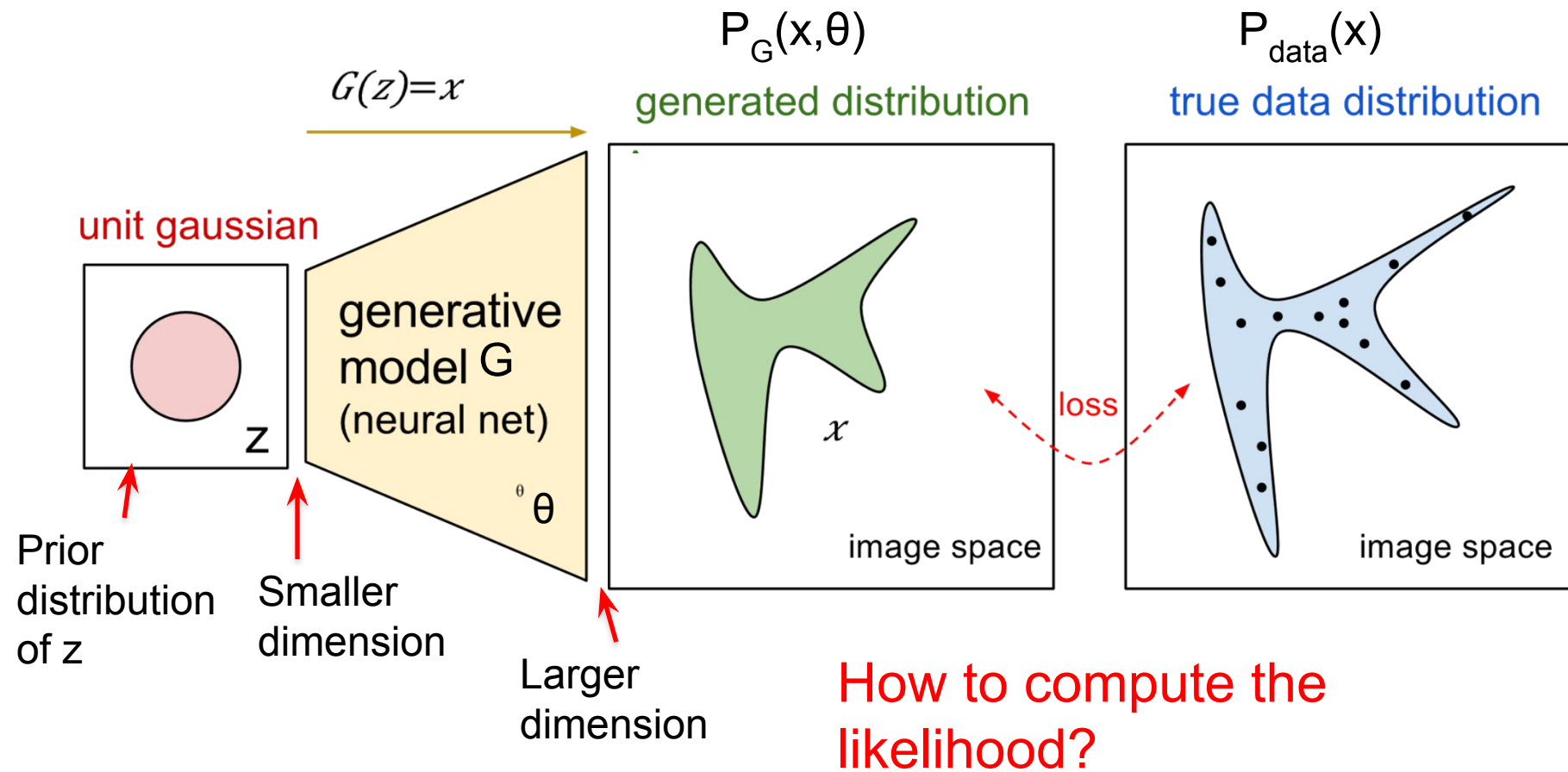
- **Minimax Loss**

- Approach based on game theory
- Two networks:
 1. Generator $g(\mathbf{z}; \mathbf{W}_g) \rightarrow \mathbf{x}$
 2. Discriminator $d(\mathbf{x}; \mathbf{W}_d) \rightarrow \Pr(\mathbf{x} \text{ is real})$

- Objective

$$\begin{aligned} \min_{\mathbf{W}_g} \max_{\mathbf{W}_d} \sum_n \log \Pr(\mathbf{x}_n \text{ is real}; \mathbf{W}_d) + \log \Pr(g(\mathbf{z}_n; \mathbf{W}_g) \text{ is fake}; \mathbf{W}_d) \\ \equiv \min_{\mathbf{W}_g} \max_{\mathbf{W}_d} \sum_n \log d(\mathbf{x}_n; \mathbf{W}_d) + \log (1 - d(g(\mathbf{z}_n; \mathbf{W}_g); \mathbf{W}_d)) \end{aligned}$$

NN for $P_G(x; \theta)$

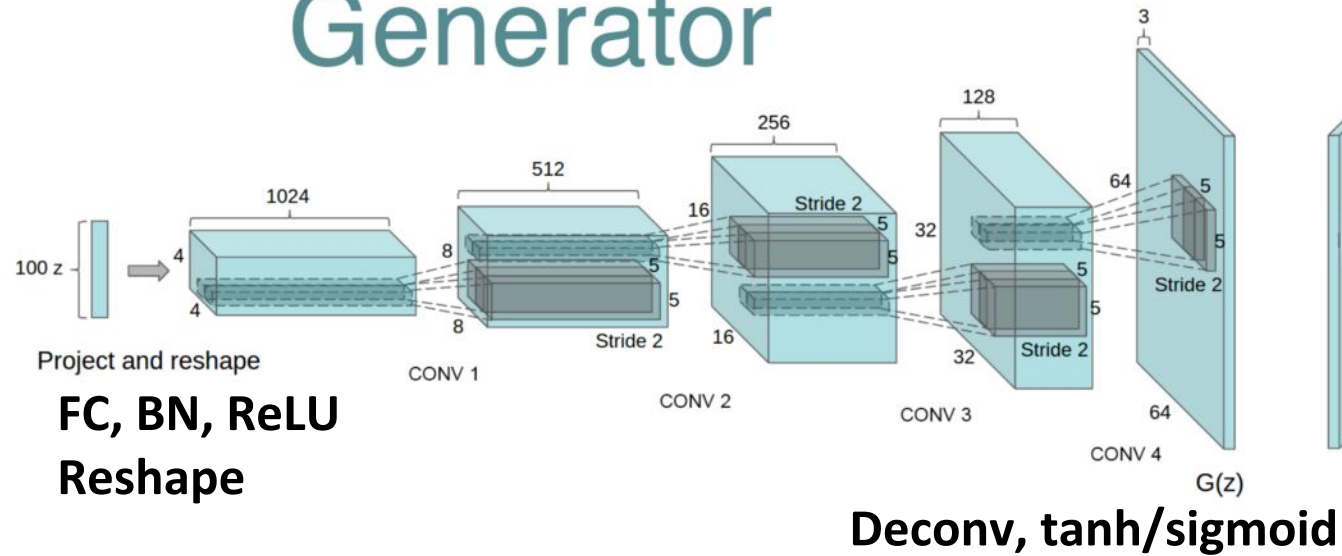


$$P_G(x) = \text{Integration}_z P_{\text{prior}}(z) I_{[G(z)=x]} dz$$

<https://blog.openai.com/generative-models/>

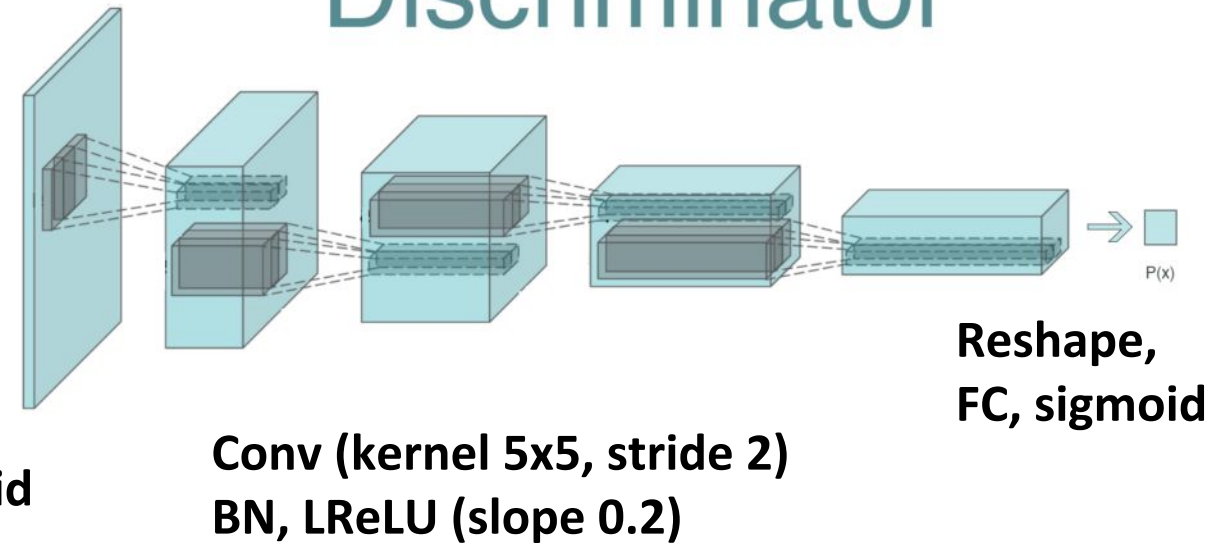
Generative Adversarial Network (GAN)

Generator

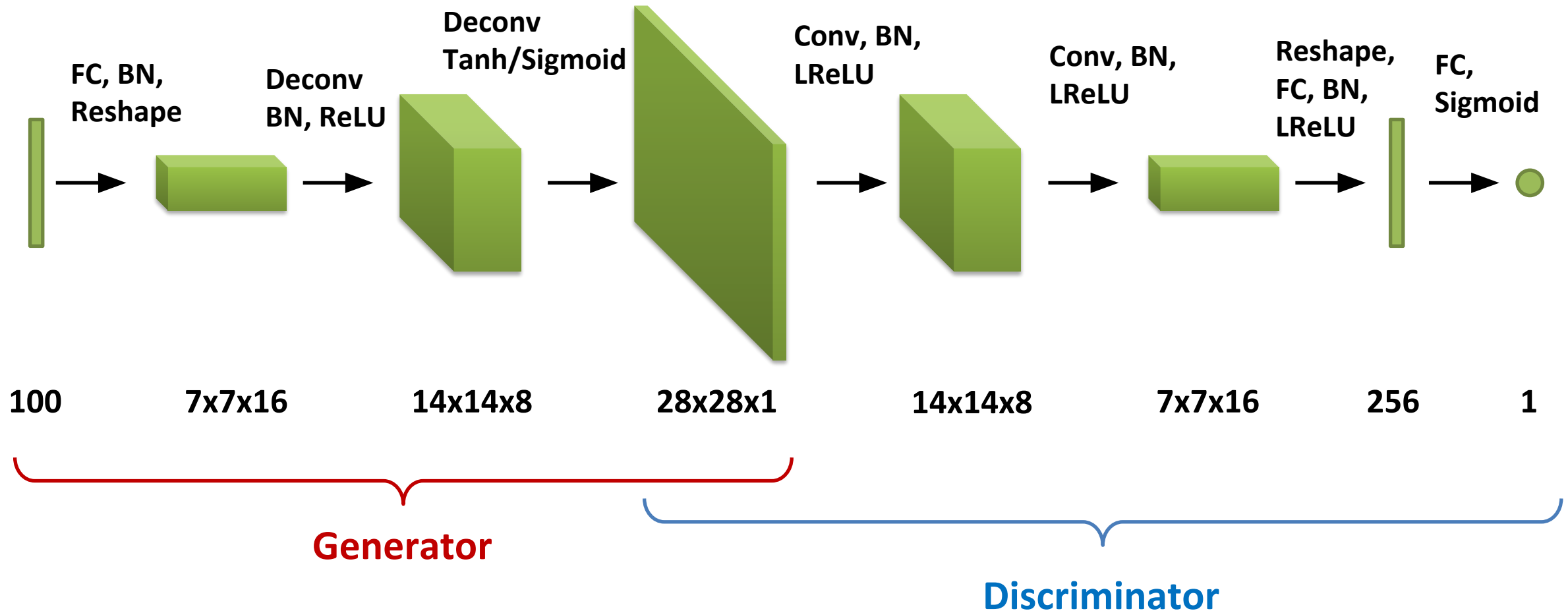


Deconv (kernel 5x5, stride 2)
BN, ReLU

Discriminator



GAN on MNIST



Issues: Convergence

- As the generator improves with training, the discriminator performance gets worse
 - Discriminator can't differentiate between real and fake.
 - If the generator succeeds perfectly, then the discriminator has a 50% accuracy.
 - Starts random guess (Less meaningful)
- **Warning:** The generator may start to train on random feedback, and its own quality may reduce.

Problems

- **Imbalance**

- One network may dominate the other
- e.g. discriminator may always tells the fake and real data and generator may not be able to fool the generator

- **Local Convergence**

- We may stuck at a local minima and not the global error
- The **discriminator feedback gets less meaningful over time.**
- The generator starts to train on junk feedback, and its own quality may collapse.

Advantages of GANs

- **Plenty of existing work on Deep Generative Models**
 - Boltzmann Machine
 - Deep Belief Nets
 - Variational AutoEncoders (VAE)
- **Why GANs?**
 - Sampling (or generation) is straightforward.
 - Training doesn't involve Maximum Likelihood estimation.
 - Robust to Overfitting since Generator never sees the training data.
 - Empirically, GANs are good at capturing the modes of the distribution.

Thank You 😊